

Laborator 9 – Crearea de grafice într-o aplicație ASP.NET Core MVC

1. In laboratorul curent vom dezvolta solutia realizata in laboratoarele anterioare Nume_Prenume_Lab4. Vom vrea un brach nou Lab4_partea_4(based on Lab4_partea 3)
2. Vom crea o pagină Dashboard unde vom afisa sub forma grafica statistici bazate pe istoricul predicțiilor:

- **Numărul total de predicții efectuate** (total în tabela PredictionHistory).
- **Prețul mediu per tip de plată** (grupare și media prețurilor pentru CASH, CARD etc.).
- **Un grafic cu distribuția prețurilor** folosind Bootstrap si Chart.js (de exemplu, intervale de preț: 0–10, 10–20, 20–30, 30–50, >50)

3. In folderul Models vom crea un ViewModel cu denumirea DashboardViewModel si urmatorul continut:

```
public class DashboardViewModel
{
    public int TotalPredictions { get; set; }

    public List<PaymentTypeStat> PaymentTypeStats { get; set; } = new();

    public List<PriceBucketStat> PriceBuckets { get; set; } = new();
}
```

4. In continuare vom crea cele doua clase: PaymentTypeStat si PriceBucketStat necesare pentru crearea graficelor. Acestea vor avea urmatorul continut:

```
public class PaymentTypeStat
{
    public string PaymentType { get; set; } = string.Empty;
    public double AveragePrice { get; set; }
    public int Count { get; set; }
}
```

```
public class PriceBucketStat
{
    public string Label { get; set; } = string.Empty; // ex. "0-10", "10-20"
    public int Count { get; set; }
}
```

5. In PredictionController adaugam metoda actiune Dashboard() în care vom calcula numarul total de predicții (câte avem salvate in baza de date in tabelul PredictionHistory), vom calcula pret mediu per tip de plata si distributia preturilor pe intervale de pret

```
[HttpGet]
```

```

public async Task<IActionResult> Dashboard()
{
    // 1. Numărul total de predicții
    var totalPredictions = await _context.PredictionHistories.CountAsync();

    // 2. Preț mediu per tip de plată + număr de predicții per tip
    var paymentTypeStats = await _context.PredictionHistories
        .GroupBy(p => p.PaymentType)
        .Select(g => new PaymentTypeStat
        {
            PaymentType = g.Key,
            AveragePrice = g.Average(x => x.PredictedPrice),
            Count = g.Count()
        })
        .ToListAsync();

    // 3. Distribuția prețurilor pe intervale (buckets)
    // Definim intervalele: 0-10, 10-20, 20-30, 30-50, >50 (exemplu)
    var allPredictions = await _context.PredictionHistories
        .Select(p => p.PredictedPrice)
        .ToListAsync();

    var buckets = new List<PriceBucketStat>
    {
        new PriceBucketStat { Label = "0 - 10" },
        new PriceBucketStat { Label = "10 - 20" },
        new PriceBucketStat { Label = "20 - 30" },
        new PriceBucketStat { Label = "30 - 50" },
        new PriceBucketStat { Label = "> 50" }
    };

    foreach (var price in allPredictions)
    {
        if (price < 10)
            buckets[0].Count++;
        else if (price < 20)
            buckets[1].Count++;
        else if (price < 30)
            buckets[2].Count++;
        else if (price < 50)
            buckets[3].Count++;
        else
            buckets[4].Count++;
    }

    // 4. Construim ViewModel-ul
    var vm = new DashboardViewModel
    {
        TotalPredictions = totalPredictions,
        PaymentTypeStats = paymentTypeStats,
        PriceBuckets = buckets
    }
}

```

```
};

return View(vm);
}
```

6. În Views/Prediction creezi view-ul Dashboard.cshtml. Utilizam clase Bootstrap de tip card- pentru a da un aspect placut Dashboard-ului:

```
@model DashboardViewModel

@{
    ViewData["Title"] = "Dashboard predicții";
}

<h2>Dashboard predicții</h2>

<div class="row mb-4">
    <!-- Card: Total predicții -->
    <div class="col-md-4">
        <div class="card text-bg-primary mb-3">
            <div class="card-body">
                <h5 class="card-title">Număr total de predicții</h5>
                <p class="card-text display-6">@Model.TotalPredictions</p>
            </div>
        </div>
    </div>
</div>

<!-- Card: Tipuri de plată + medii -->
<div class="col-md-8">
    <div class="card mb-3">
        <div class="card-body">
            <h5 class="card-title">Preț mediu per tip de plată</h5>
            <table class="table table-sm mb-0">
                <thead>
                    <tr>
                        <th>Tip plată</th>
                        <th>Număr predicții</th>
                        <th>Preț mediu</th>
                    </tr>
                </thead>
                <tbody>
                    @foreach (var item in Model.PaymentTypeStats)
                    {
                        <tr>
                            <td>@item.PaymentType</td>
                            <td>@item.Count</td>
                            <td>@item.AveragePrice.ToString("0.00")</td>
                        </tr>
                    }
                </tbody>
            </table>
        </div>
    </div>
</div>
```



```

                <td>@item.Count</td>
                <td>@item.AveragePrice.ToString("0.00")</td>
            </tr>
        }
    </tbody>
</table>
</div>
</div>
</div>
</div>

```

<!-- Secțiune: Grafic distribuție prețuri -->

```

<div class="row">
    <div class="col-md-12">
        <div class="card mb-3">
            <div class="card-body">
                <h5 class="card-title">Distribuția prețurilor prezise</h5>
                <canvas id="priceChart" height="100"></canvas>
            </div>
        </div>
    </div>
</div>

```

@section Scripts {

<!-- Chart.js din CDN -->

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<script>

// Pregătim datele pentru grafic din Model.PriceBuckets

const labels = @Html.Raw(JsonSerializer.Serialize(Model.PriceBuckets.Select(b => b.Label)));

const dataValues =

@Html.Raw(JsonSerializer.Serialize(Model.PriceBuckets.Select(b => b.Count)));

const ctx = document.getElementById('priceChart').getContext('2d');

const priceChart = new Chart(ctx, {

type: 'bar',

data: {

labels: labels,

datasets: [{

label: 'Număr de predicții',

data: dataValues,

borderWidth: 1

}]

},

options: {

responsive: true,

scales: {

y: {

beginAtZero: true,

```

        title: {
            display: true,
            text: 'Număr predicții'
        },
    },
    x: {
        title: {
            display: true,
            text: 'Interval preț (unitatea modelului)'
        },
    },
},
});
</script>
}

```

Rulam aplicatia pentru a observa graficul creat.

Sarcina laborator:

Extindeți pagina Dashboard astfel încât utilizatorul să poată selecta un interval de dată (De la – Până la), iar toate statisticile afișate (număr total de predicții, preț mediu per tip de plată, distribuția pe intervale de preț) să fie recalculcate doar pentru predicțiile din acel interval.

Sugestii de implementare:

1. În PredictionController.Dashboard:

- adăugați doi parametri opționali: DateTime? fromDate, DateTime? toDate;
- înainte de calcule, filtrați interogarea de bază:

```
var query = _context.PredictionHistories.AsQueryable();
```

```
if (fromDate.HasValue)
```

```
    query = query.Where(p => p.CreatedAt.Date >= fromDate.Value.Date);
```

```
if (toDate.HasValue)
```

```
    query = query.Where(p => p.CreatedAt.Date <= toDate.Value.Date);
```

- folosiți query (nu _context.PredictionHistories) pentru:
 - Count(),
 - GroupBy pe PaymentType,
 - lista de prețuri.

2. În DashboardViewModel:

- adăugați proprietăți pentru a reține intervalul current:

```
public DateTime? FromDate { get; set; }
```

```
public DateTime? ToDate { get; set; }
```

3. În Dashboard.cshtml:

- adăugați un formular GET deasupra cardurilor:

```
<form method="get" asp-action="Dashboard" class="row g-3 mb-3">
  <div class="col-md-3">
    <label class="form-label">De la data</label>
    <input type="date" name="fromDate" class="form-control"
      value="@((Model.FromDate?.ToString("yyyy-MM-dd")) ?? "")" />
  </div>
  <div class="col-md-3">
    <label class="form-label">Până la data</label>
    <input type="date" name="toDate" class="form-control"
      value="@((Model.ToDate?.ToString("yyyy-MM-dd")) ?? "")" />
  </div>
  <div class="col-md-2 align-self-end">
    <button type="submit" class="btn btn-primary w-100">Filtrează</button>
  </div>
</form>
```